

AgriDabaw-3D — Data Security Assessment

Unity client, Spring Boot API and Railway deployment. Prepared 31 August 2026 against commit d2bb117 of agridabao-api and the current AgriDabao2ndVer working tree.

1. Scope and method

This is a code-level review, not a penetration test. Every statement below was checked against the source: authentication and authorisation logic, the database schema and its eight migrations, configuration and secret handling, what the Unity client stores on the device, and what leaves the system to third parties. No traffic was intercepted and no deployed instance was probed, so runtime configuration on Railway (environment variable values, database backup settings, log retention) is called out where it matters but could not be verified from here.

2. Personal data the game collects

Data	Where it lives	Notes
Email address	app_user.email, unique	Also cached on the device inside the saved session
Password	app_user.password_hash	BCrypt strength 12. The plaintext is never stored or logged
Display name	app_user.display_name	Player-chosen; the only identity shown to other players
Date of birth	app_user.date_of_birth	Collected at sign-up. No feature in the codebase reads it back
Chat messages	chat_message.body	Plaintext, retained indefinitely, friends-only
Farm data	farm_save.snapshot (JSONB)	Gameplay state, plus the chosen Davao district
Device identifier	player_presence.device_id	A random GUID generated by the game, not a hardware id

3. How data moves

There are three outbound paths from the player's device, all over HTTPS:

Path	Carries	Third party
Game to API	Everything above	Railway (hosting and managed Postgres)
API to Google	Farm state, crop stats, weather, district, and the free text the player types to the adviser	Google Generative Language API
API to Resend	Email address and a one-time code	Resend (transactional email)

The Unity client also calls ISRIC SoilGrids directly for soil data by latitude and longitude. That request carries the chosen map coordinates and no account data.

No account PII reaches Google. This was verified rather than assumed: the three payload builders send simulation state only, and the sole matches for 'displayName' are crop display names such as Mangosteen. The player's own typed question does go to Google, and can of course contain anything they choose to write.

4. Controls that are in place

Layer	Control	Evidence
Transport	HTTPS end to end; the client detects a cleartext base URL and warns on Android, where API 28+ blocks it by default	ApiConfiguration.IsCleartext
Passwords	BCrypt strength 12, and the emailed codes are hashed the same way rather than stored in the clear	SecurityConfig, VerificationService
Sign-in	Two-factor by default: password plus a 9-digit emailed code on every login. The account is not created until the code is confirmed	AuthService.requestLogin / verifyRegister
Code abuse	10-minute expiry, 5 wrong attempts destroys the code, 30-second resend cooldown, one live code per email and purpose	VerificationService
Sessions	Stateless HS256 JWT. The signing secret is rejected at boot if it is not at least 32 bytes	SecurityConfig.jwtSecretKey
Authorisation	Every endpoint derives identity from the token subject, never from a client-supplied id. Checked across all nine controllers	jwt.getSubject() in 9 of 9
Data exposure	Player profiles return display name, district, money and online status only — no email, no date of birth	PlayerProfileResponse
Messaging	Chat is refused unless both players are friends	ChatService.requireFriend
Economy	Marketplace and trading are server-authoritative, with row locks and an optimistic revision check on every farm write	MarketplaceService, FarmService
SQL	All access is JPA or parameterised @Query. No string-built SQL anywhere in the codebase	grep over src/main/java
Secrets	Database, JWT, Gemini and mail credentials are environment variables with no committed defaults. The Gemini key was deliberately moved off the client, because a Unity APK ships its Inspector values inside the package	application.yml, GeminiClient
Schema	Hibernate runs with ddl-auto: validate, so the schema is owned by Flyway and the app refuses to start against a mismatched database	application.yml
Ops surface	Actuator exposes health and info only	management.endpoints
Concurrency	One account may only be played on one device at a time; a second device is refused and the first is returned to the menu	PresenceService, SessionTakeover

5. Findings

Ranked by risk to player data. None of these are exploited by anything in the codebase today; they are weaknesses an attacker could work with.

HIGH	1. A 30-day token, stored in the clear, that cannot be revoked The access token now lasts 30 days so players stay signed in, and it is kept in PlayerPrefs — an XML file in the app's private directory on Android, the registry on Windows. Nothing on the server can invalidate it: Logout only deletes the phone's copy. Anyone who extracts the file from a rooted device, a backup, or a shared PC profile holds a working session for up to 30 days, and the real owner has no way to end it. This is a deliberate trade for the convenience of staying signed in, but it is the single largest exposure in the system.
HIGH	2. No rate limiting on any endpoint Nothing throttles requests. The password step of login can be attempted without limit, and the code endpoints can be used to send repeated emails to an address. The 5-attempt cap on a verification code helps but is per code, not per account or per address, and requesting a fresh code resets it — subject only to the 30-second cooldown. The 9-digit code is large enough that guessing is not the concern; credential stuffing against the password step, and using the service to send mail, are.
MEDIUM	3. Sign-up confirms whether an email is registered Registration answers 'An account with this email already exists', which lets anyone test an address for membership. Login is written correctly by contrast, returning 'Invalid email or password' for both cases.
MEDIUM	4. Email and date of birth are cached on the device The saved session stores the whole user record, so the email address and date of birth sit in plaintext PlayerPrefs alongside the token. Only the token is needed to stay signed in; the rest is convenience.
MEDIUM	5. No way to delete an account or export its data A farm can be deleted (DELETE /api/farms/me), but there is no path to erase the account itself, the email, or the date of birth — not from the game and not from the API. For a project collecting personal data from Philippine users, the Data Privacy Act (RA 10173) grants rights of erasure and access that nothing here currently satisfies.
MEDIUM	6. Date of birth is collected but never used No code reads date_of_birth after sign-up. It is the most sensitive field collected and the easiest to stop collecting; if it is there for an age gate, the gate is not implemented.
LOW	7. Verification codes are logged when the dev flag is on With APP_VERIFICATION_EXPOSE_CODE=true the server logs each code in plaintext and returns it in the response. It defaults to false and must stay false in production. This one is worth checking in the Railway variables rather than assuming, since it cannot be verified from the source.
LOW	8. Chat is stored in plaintext and never expires Messages are kept indefinitely and readable by anyone with database access. Reasonable for a game; worth a retention limit if the player base grows.
LOW	9. No CORS policy Harmless today, because only a native client calls the API and browsers are not involved. It would become relevant the moment a WebGL build or a web dashboard is added.
LOW	10. Players can type anything into the adviser Free text is forwarded to Google. Nothing stops a player pasting personal details into it, and nothing warns them not to.

6. Recommendations, in order

#	Action	Effort
1	Verify APP_VERIFICATION_EXPOSE_CODE is false in the Railway production variables, and that APP_JWT_SECRET_BASE64 is a fresh random value never committed anywhere	Minutes
2	Add rate limiting to the four auth endpoints — per IP and per email. A simple in-memory counter would close most of finding 2	Half a day
3	Stop caching the email and date of birth on the device; keep only the token and re-fetch the profile from /api/users/me	An hour
4	Make registration answer the same way for a taken address as for a free one, sending a 'someone tried to register with your email' notice instead	An hour
5	Add account deletion that removes the user row and everything cascading from it, reachable from the game	Half a day
6	Drop date of birth from sign-up unless it is needed for an age gate you intend to build	An hour
7	Add a token version column to app_user, include it as a claim, and reject tokens that do not match. This makes Logout revoke everywhere and answers finding 1 without giving up the 30-day session	A day
8	Add a line to the adviser panel telling players not to type personal information into it	Minutes

7. Overall

The fundamentals are in better shape than is typical for a student project. Passwords are hashed properly, every endpoint takes identity from the token rather than trusting the client, the economy is server-authoritative, there is no SQL injection surface, secrets are held in environment variables, and player profiles were deliberately built not to leak email addresses. Moving the Gemini key off the client and adding two-factor sign-in were both real security decisions rather than accidents.

The gaps cluster in two places. First, session handling: a long-lived token that nothing can revoke, kept in clear storage. Second, the absence of the operational controls that a live service needs but a prototype rarely has — rate limiting, account deletion, and data minimisation. Items 1 to 4 in the table above are worth doing before the game is put in front of real players; item 7 is the one that turns the session design from acceptable into sound.

Assessed by code review only. Runtime configuration on Railway, database backup and retention policy, and Railway's own access controls were out of scope and should be confirmed separately.